Capture the Flag 3: Data-Driven UI

🏴 Flag 0 — Create Your Feature Branch

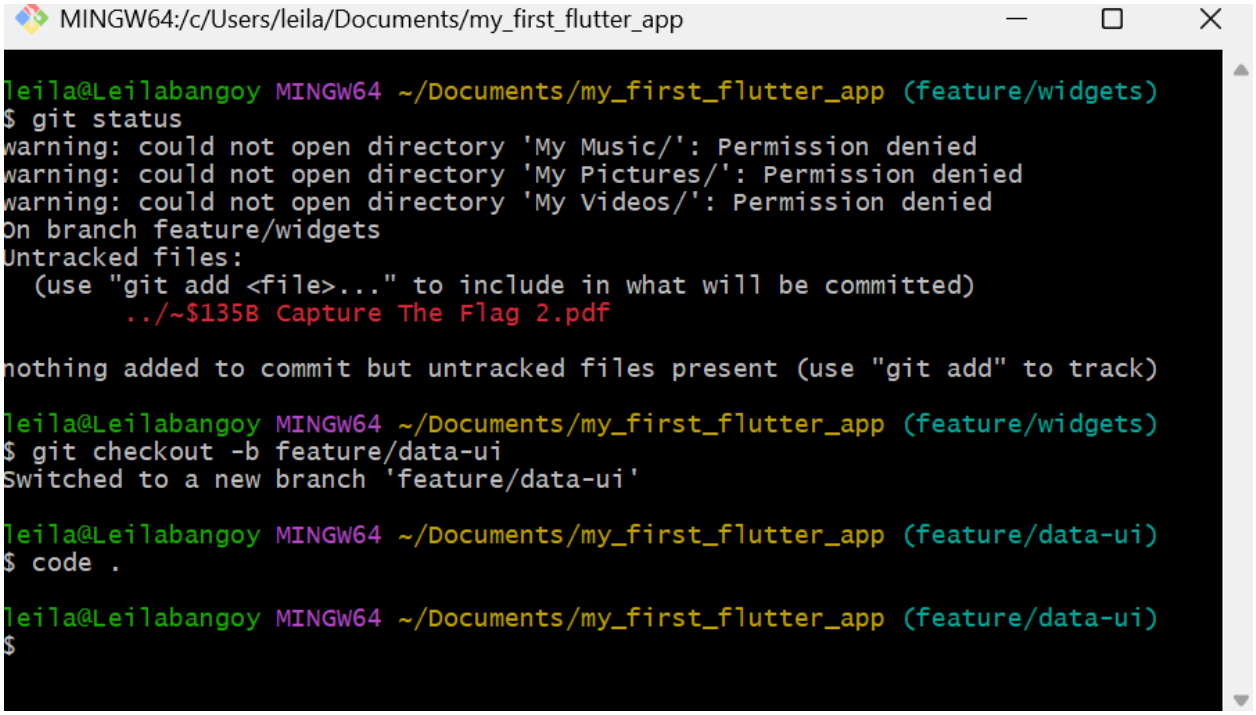
Objective: Create a feature branch before modifying your application.

Create:

`feature/data-ui`

Screenshot:

- Terminal showing your current branch



🏴 Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least **four different types** that could be used in your application.

Examples:

String
int
double
bool

Create a table identifying the information and its appropriate data type.

Create:

- Your identified data
- The corresponding data type

IDENTIFIED DATA	DATA TYPE
Name = Leila G. Bangoy	String
My First Flutter Application	String
August 11, 2026	Date Time
I Love Watching Movies and Dramas	String
20	Int
Height	Double
IsStudent	bool
Birthday : August 22, 2005	Date Time
Favorite Game: BaseBall	String
Favorite Food:Buffalo Chicken	String
Favorite Movies: Blind Side	String
Music: All Taylor Swift	String
Hobby: Watching Movies	String

🏴 Flag 2 — Create Your Data

Objective: Create appropriately typed variables for your profile.

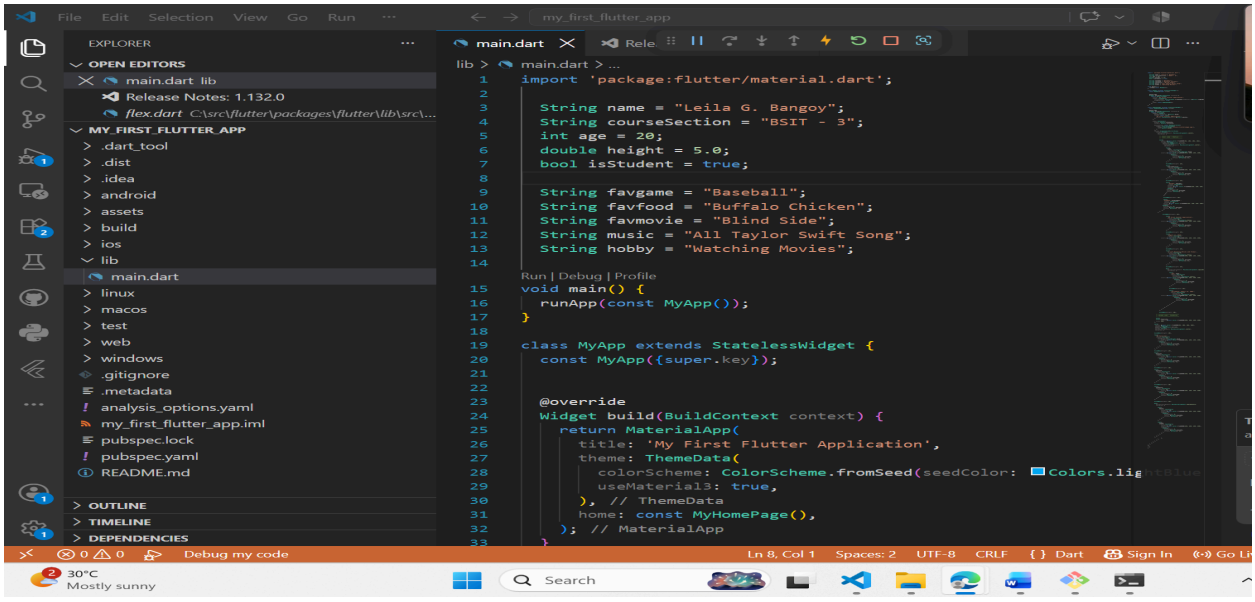
Create variables for at least:

- Name
- Course & Section
- Age
- Hobby
- Height
- Student status

Your data should exist **separately from your UI widgets**.

Screenshot:

- Your variables
- Their data types
- Application still running



🏴 Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

Your profile should now display information using the variables you created.

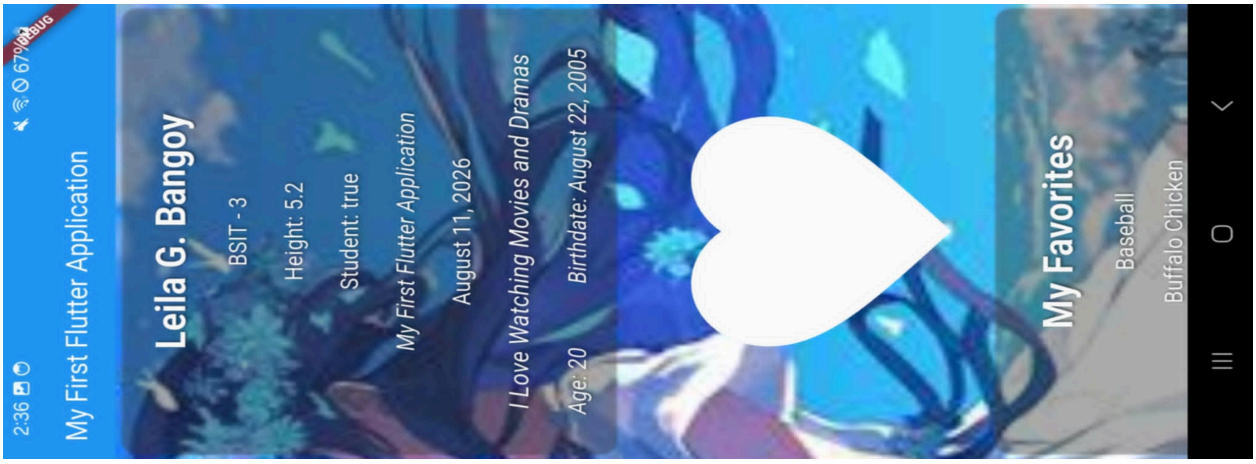
Test:

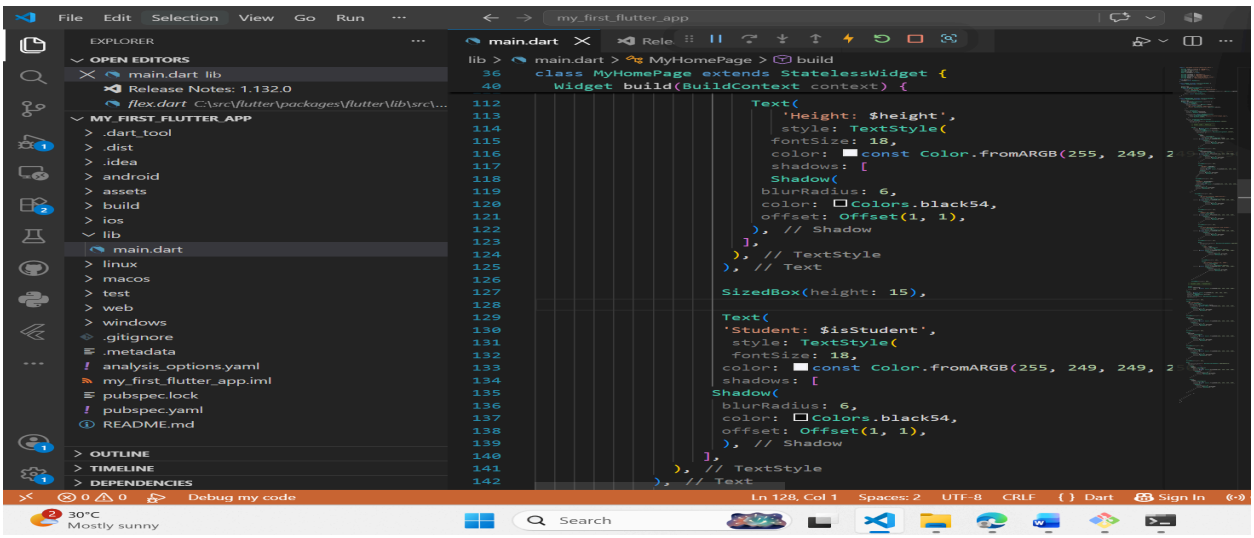
Change the value of one of your variables.

The UI should automatically display the new value.

Screenshot:

- Variables
- Widgets displaying the variables
- Updated application





🏴 Flag 4 — Make the Image Data-Driven

Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

Use the variable when displaying the image.

Then change the variable to another registered image.

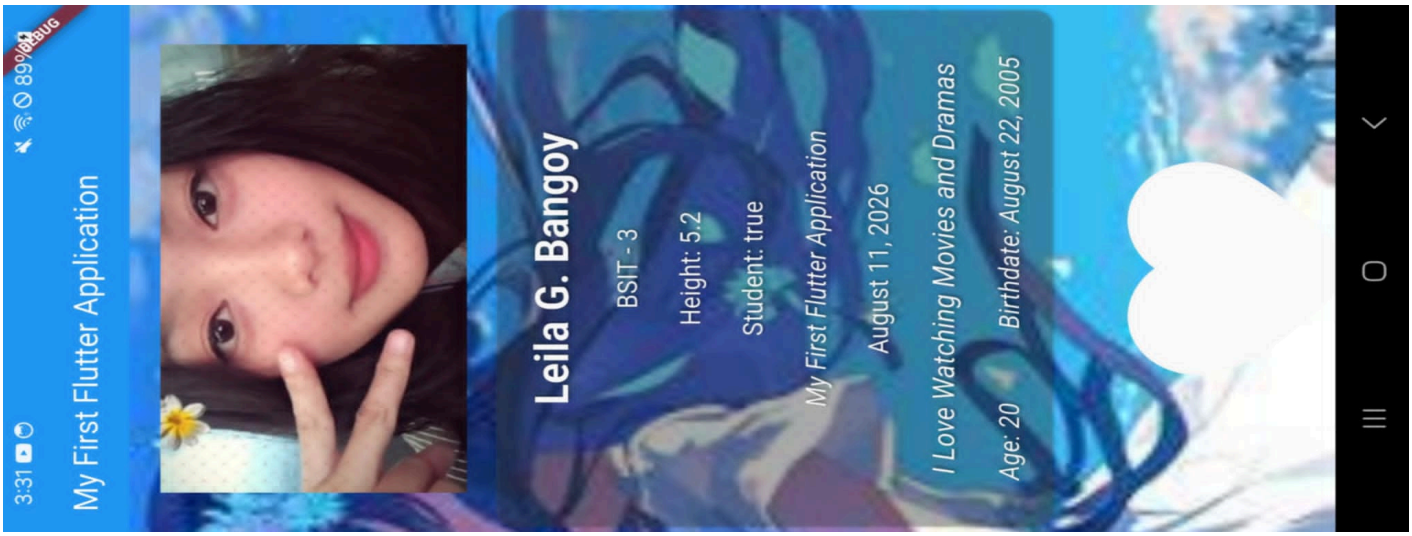
The image displayed by the application should change.

Screenshot:

- Image variable
- Image widget using the variable
- Updated application

```dart
main.dart > main
import 'package:flutter/material.dart';

  String name = "Leila G. Bangoy";
  String courseSection = "BSIT - 3";
  int age = 20;
  double height = 5.0;
  bool isStudent = true;

  String profileImage = 'assets/Profile.jpg';

  String favgame = "Baseball";
  String favfood = "Buffalo Chicken";
  String favmovie = "Blind Side";
  String music = "All Taylor Swift Song";
  String hobby = "Watching Movies";
```

```dart
          // ========================
          // FIRST CARD - PROFILE
          // ========================
            Center(
            child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Padding(
             padding: EdgeInsets.only(top: 20),
            child: Image.asset(
            profileImage,
            width: 350,
            height: 200,
            fit: BoxFit.cover,
          ), // Image.asset
        ), // Padding

        SizedBox(height: 15),
```



🏴 Flag 5 — Five Profiles, One Design

Objective: Create at least **five different profile cards** using the same visual design.

Each profile must contain at least:

- Image
- Name
- Course & Section
- Age
- Hobby

The five profiles must contain different information.

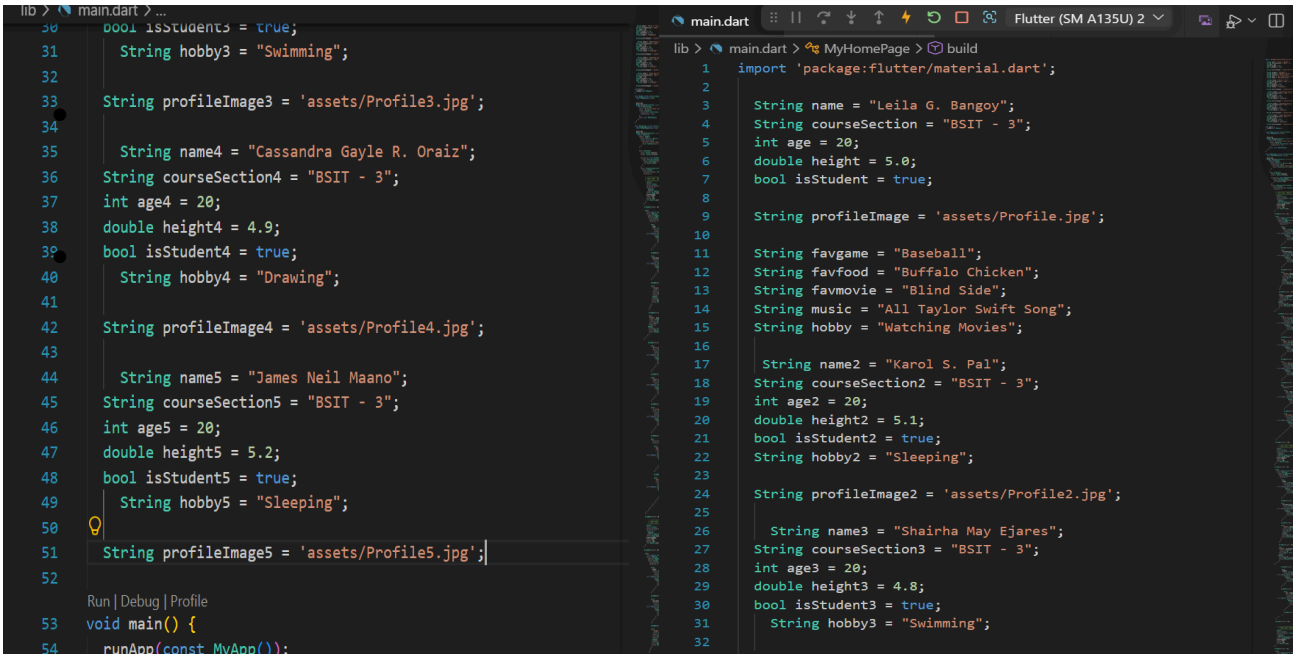
Important

All five profiles should use the **same visual design**.

You are demonstrating that the same UI can represent different data.

Screenshot:

Your 5 different profile data



```
lib > main.dart > ...
30    bool isStudent3 = true;
31      String hobby3 = "Swimming";
32
33    String profileImage3 = 'assets/Profile3.jpg';
34
35      String name4 = "Cassandra Gayle R. Oraiz";
36    String courseSection4 = "BSIT - 3";
37    int age4 = 20;
38    double height4 = 4.9;
39    bool isStudent4 = true;
40      String hobby4 = "Drawing";
41
42    String profileImage4 = 'assets/Profile4.jpg';
43
44      String name5 = "James Neil Maano";
45    String courseSection5 = "BSIT - 3";
46    int age5 = 20;
47    double height5 = 5.2;
48    bool isStudent5 = true;
49      String hobby5 = "Sleeping";
50
51    String profileImage5 = 'assets/Profile5.jpg';
52
      Run | Debug | Profile
53  void main() {
54    runApp(const MyApp());
```

```
main.dart                                   Flutter (SM A135U) 2
lib > main.dart > MyHomePage > build
1   import 'package:flutter/material.dart';
2
3     String name = "Leila G. Bangoy";
4     String courseSection = "BSIT - 3";
5     int age = 20;
6     double height = 5.0;
7     bool isStudent = true;
8
9     String profileImage = 'assets/Profile.jpg';
10
11    String favgame = "Baseball";
12    String favfood = "Buffalo Chicken";
13    String favmovie = "Blind Side";
14    String music = "All Taylor Swift Song";
15    String hobby = "Watching Movies";
16
17     String name2 = "Karol S. Pal";
18    String courseSection2 = "BSIT - 3";
19    int age2 = 20;
20    double height2 = 5.1;
21    bool isStudent2 = true;
22    String hobby2 = "Sleeping";
23
24    String profileImage2 = 'assets/Profile2.jpg';
25
26      String name3 = "Shairha May Ejares";
27    String courseSection3 = "BSIT - 3";
28    int age3 = 20;
29    double height3 = 4.8;
30    bool isStudent3 = true;
31      String hobby3 = "Swimming";
32
```

🏴 Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to **scroll through content that is larger than the available screen**.

Research Flutter's **scrolling widgets** and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

Requirement:

The user must be able to:

Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

without changing the size of the screen.

Screenshot:

- The scrolling widget in your code
- Top portion of the application
- The rest of the portion of the application showing the later profiles

🏴 Flag 7 — Survive the Database

Objective: Prepare your UI for incomplete data.

Now imagine your profiles aren't manually created.

Someday, the data will come from a database.

And real data isn't always perfect.

Some information may be:

- Missing
- Empty
- `null`
- Not provided

Create at least **three profiles with missing information**.

For example:

Profile A
Name: Maria
Course: BSIT
Hobby: missing

Profile B
Name: Juan
Course: missing
Hobby: Gaming

Profile C
Name: missing
Course: MMA
Hobby: Drawing

Your application must **not crash** when information is missing.

Instead, provide an appropriate fallback.

Examples:

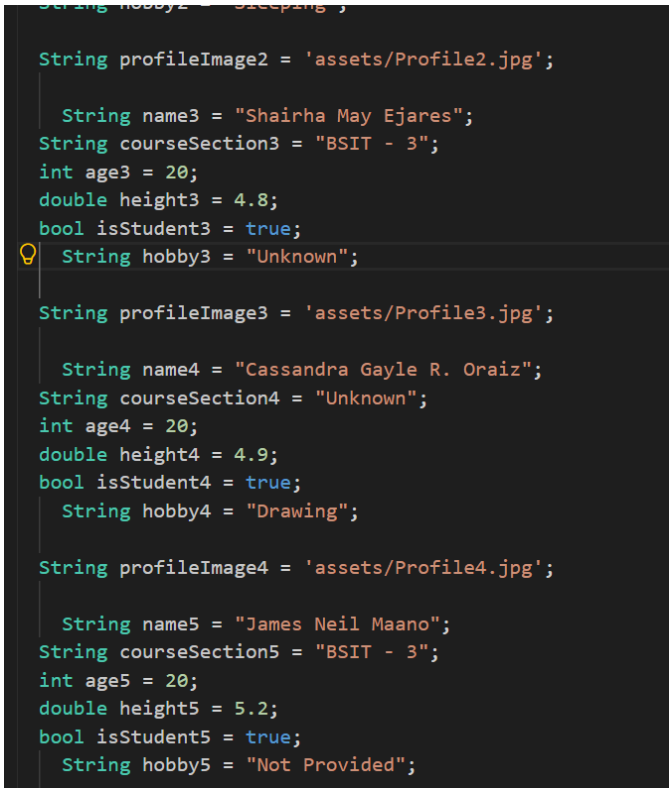
Hobby: Not provided
Course: Unknown

Challenge

Your placeholders should look intentional and clean.

Screenshot:

- Incomplete profile data
- Application displaying fallback values
- Application still functioning correctly



```
String profileImage2 = 'assets/Profile2.jpg';

  String name3 = "Shairha May Ejares";
String courseSection3 = "BSIT - 3";
int age3 = 20;
double height3 = 4.8;
bool isStudent3 = true;
  String hobby3 = "Unknown";

String profileImage3 = 'assets/Profile3.jpg';

  String name4 = "Cassandra Gayle R. Oraiz";
String courseSection4 = "Unknown";
int age4 = 20;
double height4 = 4.9;
bool isStudent4 = true;
  String hobby4 = "Drawing";

String profileImage4 = 'assets/Profile4.jpg';

  String name5 = "James Neil Maano";
String courseSection5 = "BSIT - 3";
int age5 = 20;
double height5 = 5.2;
bool isStudent5 = true;
  String hobby5 = "Not Provided";
```

🏴 Flag 8 — Save Your Progress

Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/data-ui`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit